



Lecture 15

Microservice Deployment

Sharif University of Technology
Winter 2026



Learning Objectives

Understand how logical microservice designs map to physical deployments

Know why multiple instances, load balancers & multi-zone deployments matter

Explain database sharing rules — instances vs. services

Apply 5 core microservice deployment principles in system design

Compare physical machines, VMs, containers and PaaS/FaaS options

Design zero-downtime and desired-state deployments



Part I

Logical → Physical

What actually happens when you deploy microservices?

The Deceptive Simplicity of Logical Diagrams



Logical View



Looks simple!
"Invoice talks to Order"

Physical Reality

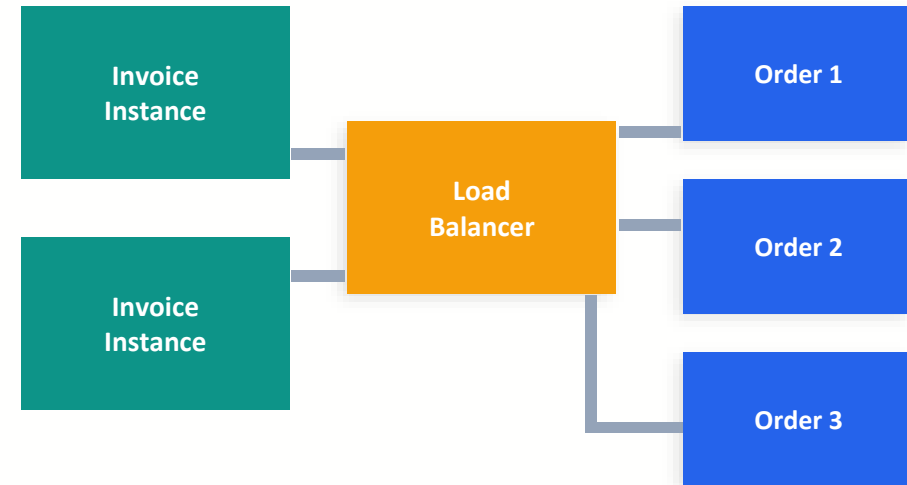
- Multiple instances per service
- Load balancers for traffic routing
- Databases with read replicas
- Multiple data centers / zones
- Network latency & failures
- Different environments (dev/CI/prod)
- Scaling policies & auto-healing

Multiple Instances



Why run multiple instances?

- Handle more load (horizontal scaling)
- Survive failure of a single instance
- Zero-downtime rolling upgrades
- Geographic distribution



Rule of Thumb

In production, never run fewer than 2 instances of a service. Single instances are single points of failure.

Distributing Across Data Centers / Availability Zones



Cloud providers do NOT guarantee single-machine or single-zone SLAs

Data Center 1 / Zone A

Invoice A

Order A1

Order A2

Data Center 2 / Zone B

Invoice B

Order B1

Order B2

Real-World Context

- AWS, Azure, GCP do not SLA single VMs
- Availability Zones $\approx$ independent data centers
- Deploy across ≥ 2 zones for true HA
- Region failures do happen
- Cloud-managed services (RDS, etc.) handle failover

Design Challenge: Database Per Instance?



Design Question

You have 3 instances of the Order service. Should each instance have its own separate database?

What happens if a customer places an order via Instance A and then queries it through Instance B?

Instance A

Instance B

Instance C

?

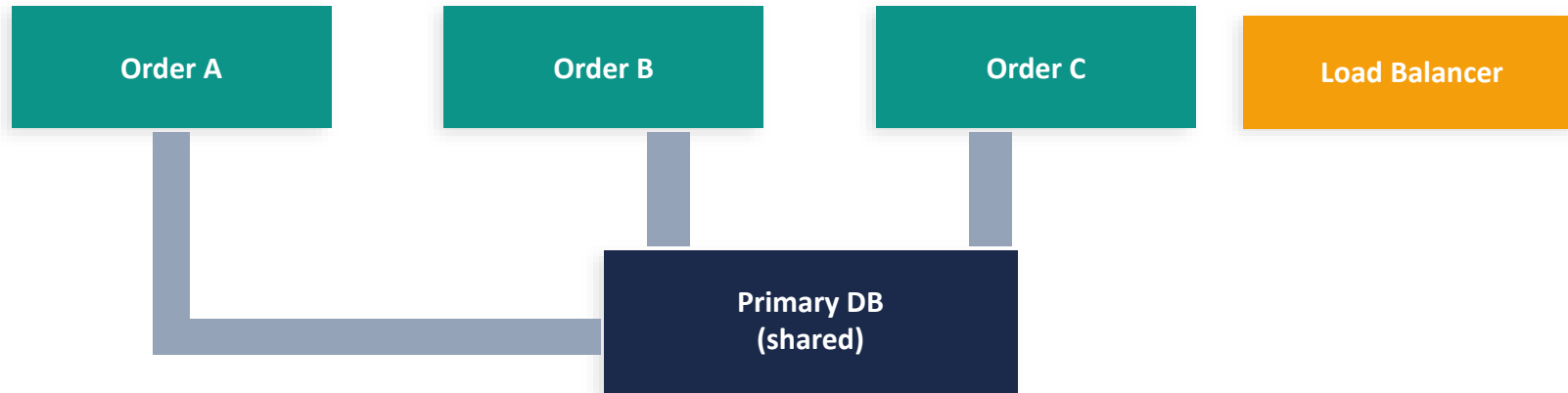
?

?

DB? DB? DB?

Answer: Shared Database Within a Logical Service

☒ No – instances of the SAME service share ONE database.



Does this violate 'Don't share databases'? No! That rule means DON'T share across DIFFERENT services. Sharing between instances of the SAME service is fine — logic stays in one place.

Database Scaling: Read Replicas

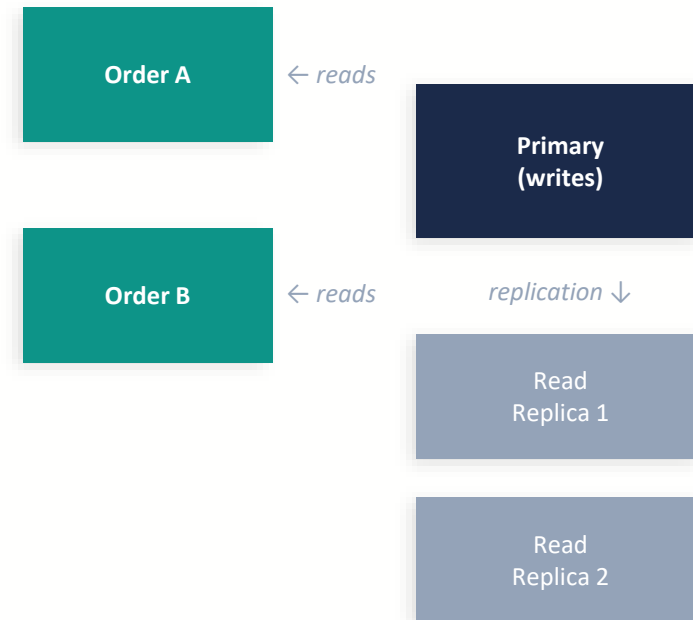


The Problem with a Single DB Node

- All reads AND writes hit one machine
- CPU, memory and I/O become bottleneck
- Scaling writes requires complex sharding
- But reads can be served from replicas!

Read Replica Pattern

- Primary node handles ALL writes
- Changes replicate to read replicas async
- Read traffic distributed across replicas
- Add more read replicas as needed
- ⚠ Slight read lag (replication delay)



Shared vs. Dedicated Database Infrastructure



Aspect	Shared DB Infrastructure	Dedicated DB per Service
Cost	✓ Lower hardware & licensing	⚠ Higher per-service cost
Management	Centralised team manages one server	Each team owns their DB
Isolation	⚠ Shared failure domain	✓ Independent failure isolation
Autonomy	⚠ Central bottleneck	✓ Teams choose own DB tech
Typical use	On-premises / cost-constrained	Cloud-native (AWS RDS, etc.)

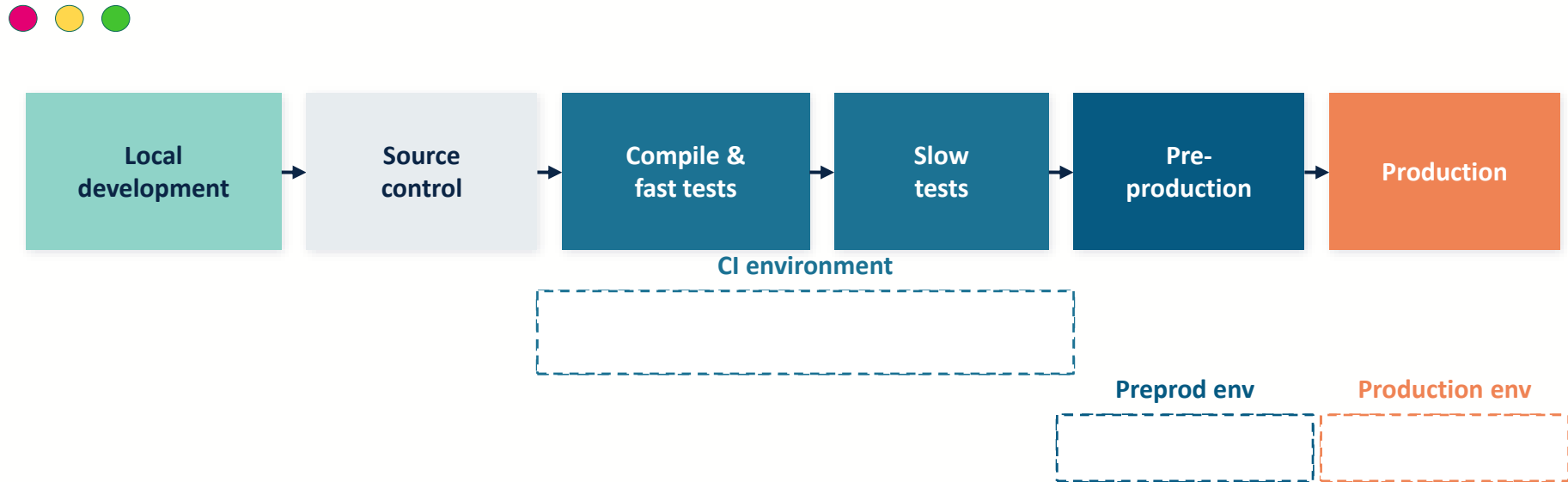
Cloud recommendation: Provision dedicated DB infrastructure per microservice (AWS RDS, Azure Database). Lower management overhead enables better isolation.

What Is an “Environment”?



- An environment is somewhere your software runs, serving a particular purpose in the development → production journey.
- Environments differ in many ways:
 - Some have production data, others don't
 - Some include ALL services; others swap out missing services for fakes/stubs
- The number and shape of environments depends entirely on how YOUR team develops and ships software — there's no fixed list.
- Typical journey: developer laptop → CI server → integrated test environment → ... → production

Deployment Environments: The Pipeline



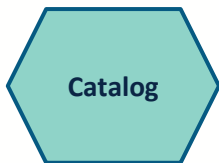
Local dev → commit triggers CI (fast + slow tests) → deploy to preproduction for manual verification (optional) → deploy to production.

Each arrow is a deployment. The microservice artifact is the same — only its configuration and topology change.

The Same Service, Three Different Topologies

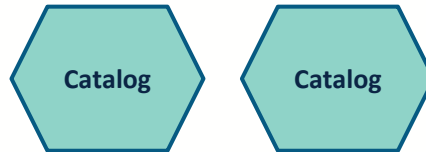


Developer laptop



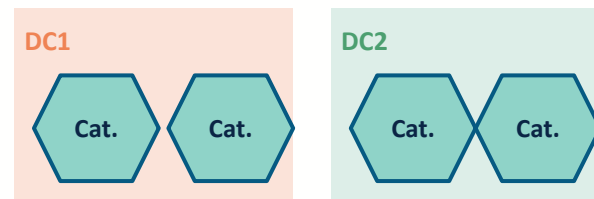
1 instance
fast build, unlike-prod hardware

CI environment



2 instances, same machine
(test LB logic, keep cost low)

Production



4 instances total,
spread across 2 data centers

← *Faster feedback*

More production-like →

Same microservice artifact — wildly different deployment topology depending on where it runs.



Part II

Principles of Microservice Deployment

5 foundational ideas every engineer must know

5 Core Deployment Principles



Isolated Execution

Each instance in its own execution environment



Infrastructure as Code

Define infra in code, stored in version control



Desired State Management

Platform self-heals to match declared state



Focus on Automation

Automate everything; people don't scale



Zero-Downtime Deployment

New versions without service interruption

Principle 1: Isolated Execution



Each microservice instance gets its OWN computing environment

✗ Problem: Multiple Services, One Host

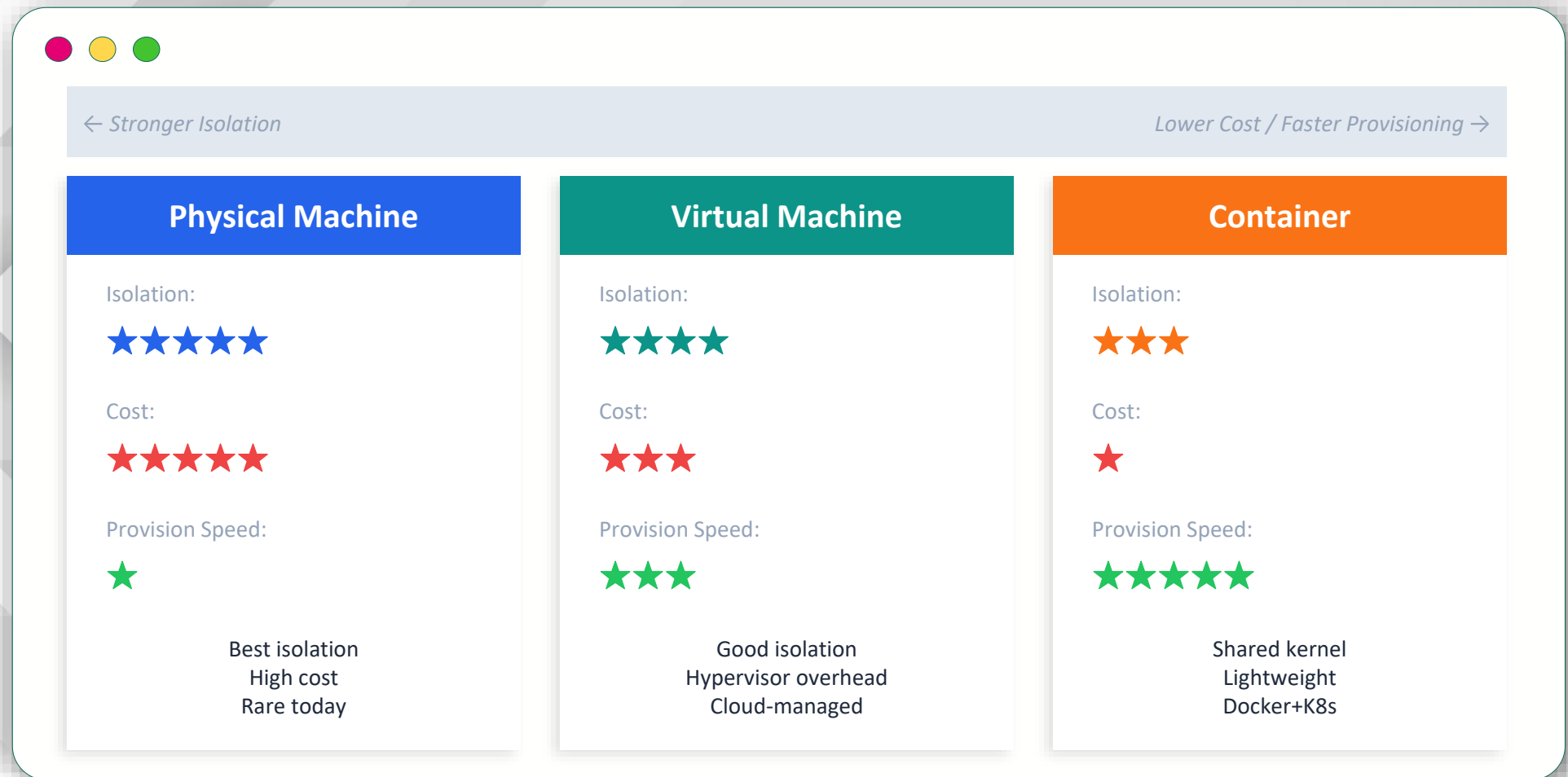
- CPU & memory contention between services
- Monitoring is ambiguous — whose CPU?
- One busy service starves others
- Contradictory dependency versions conflict
- Deployment of one affects the others
- Team autonomy undermined (central ops owns host)

✓ Solution: One Service Per Host

- Dedicated CPU, memory, I/O per instance
- Independent monitoring & alerting
- Each team fully controls their environment
- Different runtimes/versions on same hardware
- Independent deployments — no coordination needed
- Failure blast radius limited to that service

Industry example: Gilt (fashion retailer) initially shared hosts — one service's load surge impacted everything. Isolation fixed it.

Isolation Spectrum



Principle 2: Focus on Automation



- As you add microservices, you get more processes, more configuration, more instances to monitor.
- Microservices push a lot of complexity into the operational space.
- If operations stays manual, more services literally requires more people — it doesn't scale.
- What to look for in tooling:
 - Can you launch / shut down a machine with a line of code?
 - Can you deploy your software automatically, end to end?
 - Can database changes happen without manual intervention?
- Self-service provisioning keeps developers productive — they shouldn't be blocked on a centralized ops team for routine tasks.

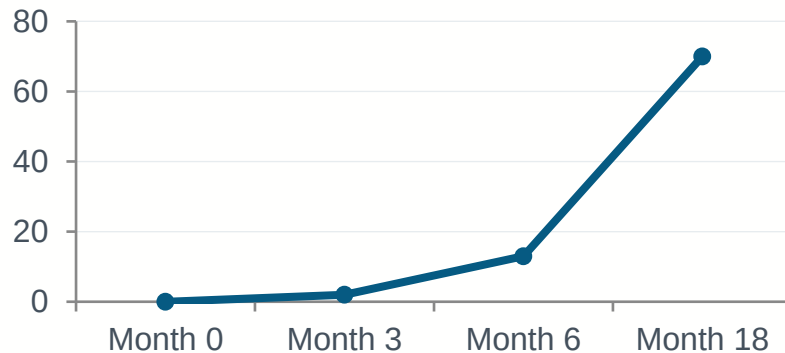
The Payoff: Two Case Studies in Automation



realestate.com.au (REA)

- First 3 months: heavy investment in tooling only 2 services shipped.
- Next 3 months: 10–15 more services
- By 18 months: 70+ services in production.

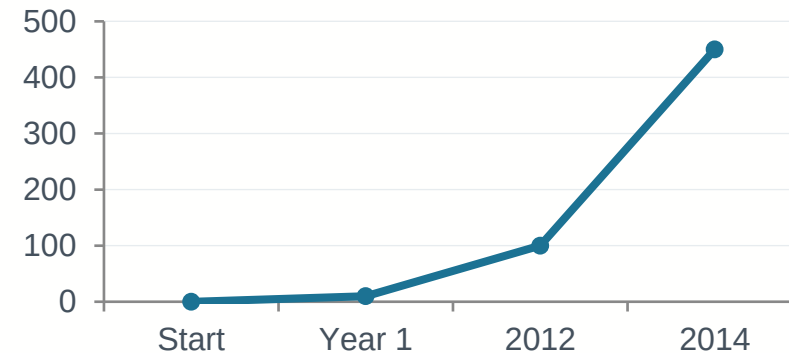
REA growth



Gilt

- Year 1 of migration: ~10 microservices live.
- By 2012: 100+. By 2014: 450+ — roughly 3 microservices per developer.
- Similar ratios seen at the Financial Times.

Gilt growth



Principle 3: Infrastructure as Code (IaC)



- Definition: configure your infrastructure using machine-readable code, kept in version control.
- Could be a Chef/Puppet recipe, or even bash scripts — the point is that running the code reliably produces a known state.
- Benefits this unlocks:
 - Version-controlled — auditors can see exactly who changed what, and when
 - Testable — infrastructure changes can go through the same review/test pipeline as application code
 - Repeatable — environments can be recreated on demand

Why IaC Matters: A Real Court Case



“Can you rebuild your production environment exactly as it was, 3 years ago?”

- The situation: As part of a court case, a client had to recreate an entire running system as of a specific date years earlier — down to OS patch levels and message broker contents.
- The reality: Infrastructure configuration was NOT in version control. The team spent over THREE MONTHS painstakingly reconstructing it by wading through old emails and release notes.
- The outcome: The court case, already long-running, was STILL not resolved by the time the author's engagement ended.

If it had been stored in version control, this would have been trivial. IaC isn't just convenience — it's institutional memory.

Principle 4: Zero-Downtime Deployment

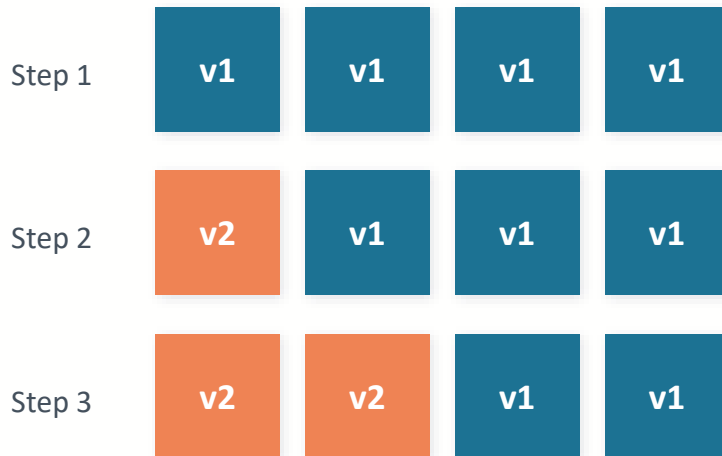


- Definition: deploying a new version of a microservice causes NO downtime for its users — human or machine.
- Without it, you may need to coordinate releases with upstream consumers to warn them of an outage — reintroducing the coordination microservices try to avoid.
- With confidence that a release won't interrupt users, teams release far more often — and can do it during normal working hours (better for quality of life AND for getting help if something goes wrong).
 - Easiest with asynchronous, message-based communication — messages simply wait until you're back up.
 - Harder with synchronous communication (e.g. REST/gRPC) — needs techniques like rolling upgrades or blue-green deployment.

Two Techniques for Zero-Downtime Releases

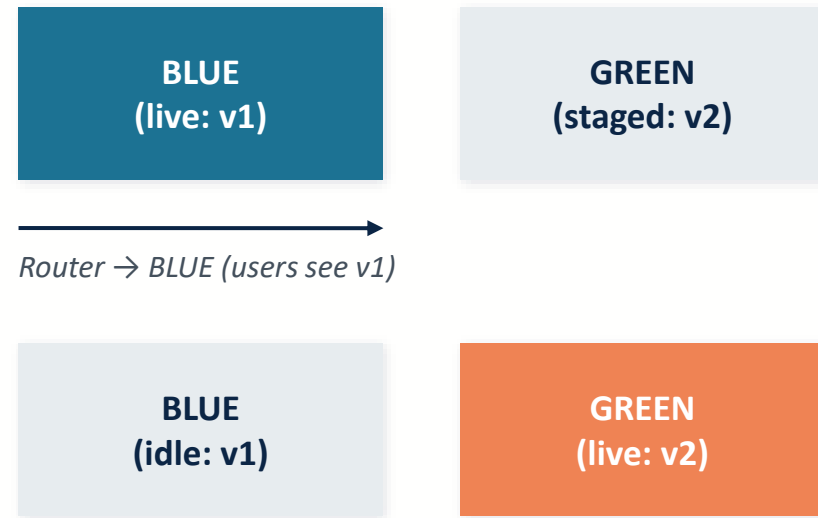


Rolling Upgrade (e.g. Kubernetes)



Old (v1) instances are gradually replaced by new (v2) instances. Traffic always has enough healthy instances to serve from.

Blue-Green Deployment



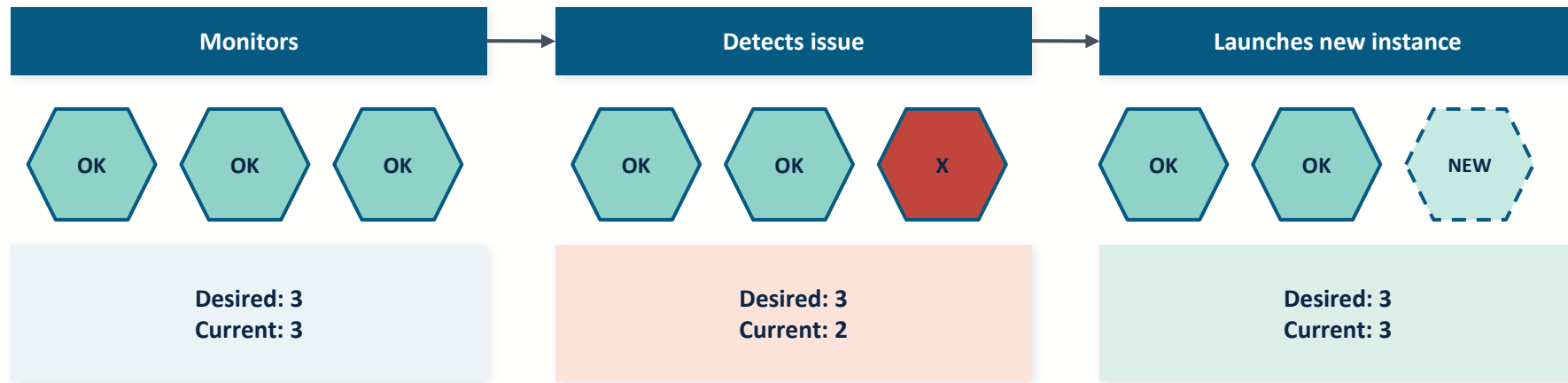
Router switches → GREEN (users see v2)
No new instances started mid-flight — just a routing switch. Instant rollback = switch back.

Principle 5: Desired State Management



Definition: you specify the infrastructure requirements for your application and the PLATFORM continuously works to keep reality matching that specification.

If reality drifts from desired state (an instance dies, hardware fails, a whole data center goes down) — the platform automatically takes action to fix it, with NO human intervention.



Examples: Kubernetes, cloud autoscaling groups (AWS/Azure)

Desired State Management: A Cautionary Tale



What happened

- Shutting down a dev cluster on AWS at the end of the day, to save money overnight.
- Killed an EC2 instance — another one popped right back up.
- Took ~15 minutes of “whack-a-mole” before realizing: an autoscaling group enforced a MINIMUM instance count.
- AWS bills per started hour — so each restart was an extra full hour of cost.

Prerequisites for desired state mgmt

- Fully automated deployment for microservice instances — the platform must be able to launch new ones itself.
- Understand instance startup time — slow startup may require spare capacity to bridge the gap.
- Recommendation: don't build your own — adopt a platform (Kubernetes, cloud autoscaling) that treats this as first-class.
- Consider waiting until you have a FEW microservices running before adopting — don't front-load too much new tech at once.

Five Principles: Recap



Principle	In One Line	Example Enabler
Isolated execution	Own resources per instance; no noisy neighbors	Containers, VMs, dedicated hosts
Focus on automation	Self-service, scriptable infra & deploys	CI/CD pipelines, cloud APIs
Infrastructure as code	Infra defined in version-controlled code	Terraform, Pulumi, CDK
Zero-downtime deployment	Releases invisible to users	Rolling upgrades, blue-green
Desired state management	Platform maintains your defined state	Kubernetes, autoscaling groups, Nomad

Next: we use these five principles as a scorecard for evaluating real deployment platforms.



Part III

Deployment Technology Options

Physical machines, VMs, containers, PaaS and FaaS

Six Ways to Run a Microservice Instance



Physical Machine

- Deployed directly on hardware, no virtualization layer

Virtual Machine

- Deployed onto a VM via a hypervisor (EC2, Azure VM, ...)

Container

- Runs as a container on a VM or physical machine, often via Kubernetes

Application Container

- Multiple service instances inside one runtime (Tomcat, IIS, ...)

Platform as a Service (PaaS)

- Push code; the platform manages servers (Heroku, App Engine, ...)

Function as a Service (FaaS)

- Deploy functions; platform triggers & scales them (Lambda, ...)

We'll move along this list from “most manual / most control” to “most automated / least control”.

Option 1: Physical Machines



What it is

- Microservice instance runs directly on hardware — no hypervisor, no container runtime between you and the metal.

Why it's fallen out of favor

- Low utilization: if you only use half a machine's CPU/RAM, the rest is wasted.
- Packing multiple services onto one physical box to fix this violates Principle 1 (isolated execution).
- IaC / desired state / zero-downtime all become much harder at the single-machine level — you need an abstraction layer on top.

Verdict: only justified by very specific constraints (e.g. extreme performance/regulatory requirements). For almost everyone, virtualization or containers are strictly better.

Option 2: Virtual Machines

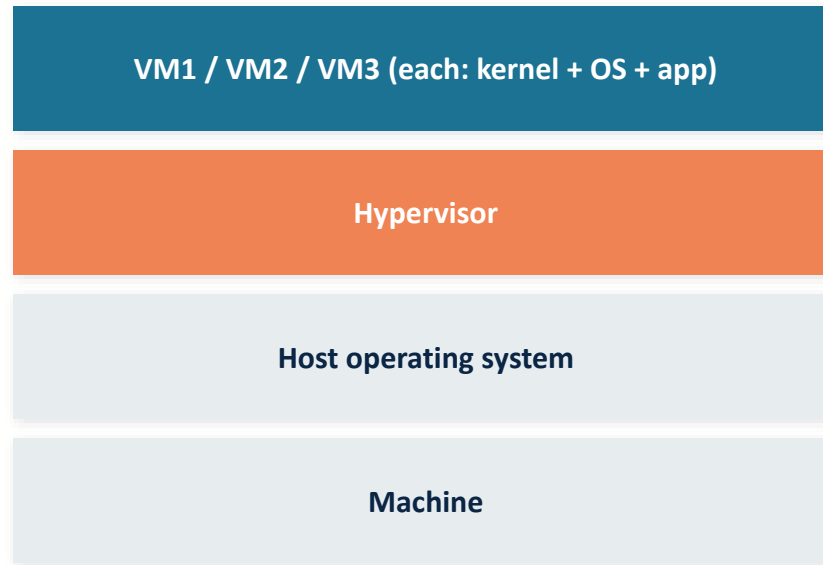


- A hypervisor splits one physical machine into multiple VMs, each acting like a normal server to the software running inside it.
- Each VM gets its own full operating system and kernel — a “hermetically sealed” environment, isolated by the hypervisor.
- You can assign CPU, memory, I/O, and storage portions to each VM independently.
- **Good isolation — but you still lose multiple instances if the underlying physical machine fails.**
- Managed VM platforms (AWS EC2, Azure VMs) are easy to automate via mature APIs, and integrate with autoscaling groups for desired state management.

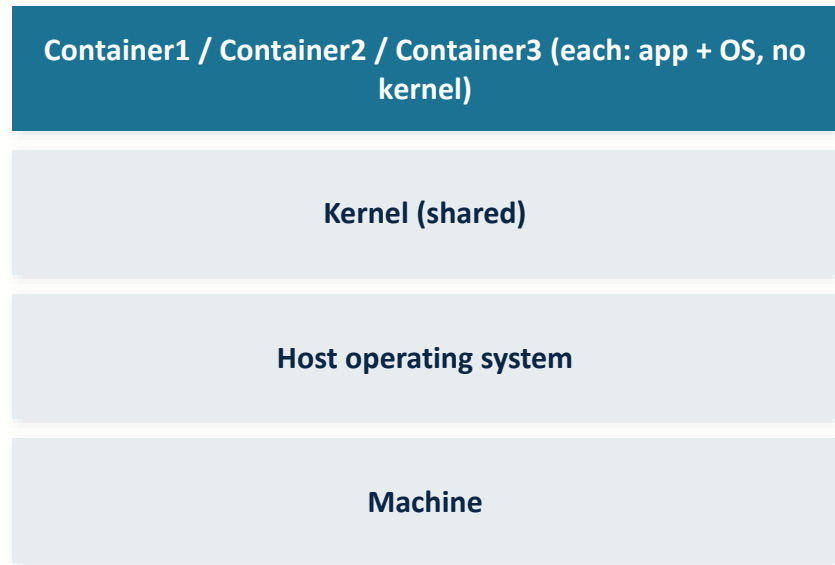
Type 2 Virtualization vs. Containers



Type 2 Virtualization (AWS, VMware)



Container-Based Virtualization



The hypervisor needs its own CPU, memory, and I/O to do its job

Containers skip the hypervisor AND the per-instance kernel entirely — they share the host kernel

The Sock Drawer Analogy



The analogy

- Think of a physical machine as a sock drawer.
- Adding wooden dividers (VMs via a hypervisor) lets you organize the drawer — but the dividers themselves take up space.
- More dividers = less total room for socks, even though each compartment is easier to manage.



In computing terms

- Pack more VMs onto a physical machine → the hypervisor needs more resources to manage them all.
- Eventually, slicing further yields diminishing returns: proportionally more resources go to overhead, less to your workloads.
- Containers have much “thinner dividers” — no hypervisor, no per-instance kernel — so a much higher proportion of the “drawer” is usable.

This is the fundamental economic reason containers became so popular for microservice workloads.

Virtual Machines: Scorecard & Case Study



Scored against our 5 principles

- Isolation: ✓✓ strong — own kernel, ring-fenced resources
- Automation: ✓ good on managed cloud VMs (rich APIs); weaker on traditional on-prem virtualization platforms (VMware) often centrally controlled
- IaC: ✓ well supported via Terraform etc.
- Zero-downtime: possible, but more manual work than container platforms
- Desired state: ✓ via autoscaling groups

Case study: Netflix

- One of the poster children for microservices.
- Built much of its microservice architecture on top of AWS managed VMs (EC2) — not containers.
- Proof that VMs can run large-scale microservice systems — to great effect — when you need strong isolation or can't containerize.

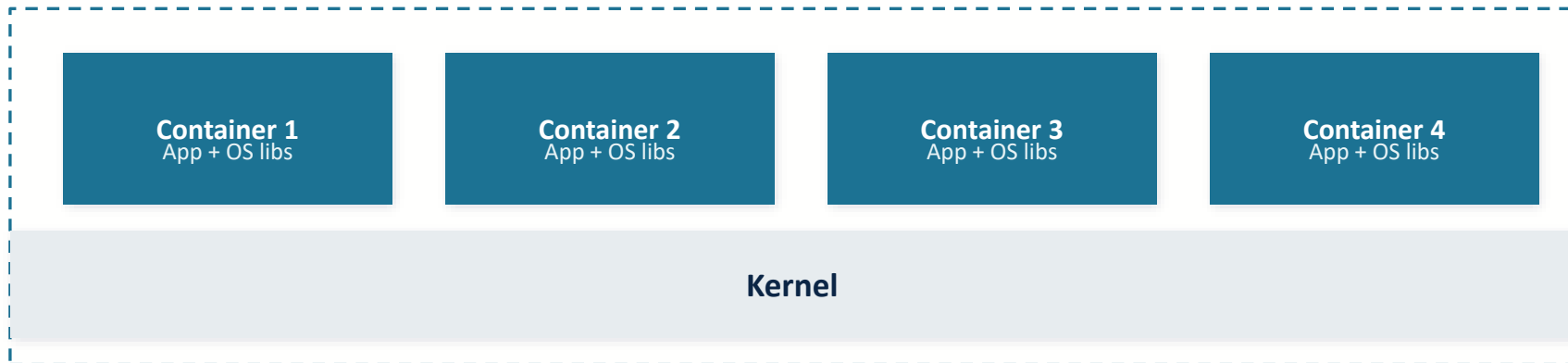
Containers are more popular today, but VMs remain a perfectly valid, production-proven choice

Option 3: Containers — “Isolated, Differently”



- Containers emerged on Linux: the kernel manages a tree of processes, assigning resources and permissions to each.
- **A container = an abstraction over a SUBTREE of that process tree, with the kernel doing the hard work.**
- Containers on the same machine normally share the SAME underlying kernel — unlike VMs, which each run their own kernel.
- This means the host could run Ubuntu while containers run CentOS — as long as both work with the same shared kernel.
- History: LXC brought this into the Linux mainstream → Docker added a much higher-level abstraction (images, registries, tooling) on top.

Shared Kernel, Separate Containers



Each container packages an app with its own OS libraries/dependencies, but ALL containers on this host share one kernel.

Benefits: startup in seconds (vs. minutes for VMs), fine-grained resource control, and — because there's no hypervisor — you can run many more containers per host than VMs (“thinner dividers”).

Common pattern: a large cloud VM (EC2 instance) running multiple containers — combining on-demand elasticity with fast, flexible containers.

Containers Aren't Perfect



⚠ Known limitations

- Networking: routing the outside world to containers needed extra tooling (early LXC) — Docker improved this hugely.
- Weaker isolation than VMs/physical machines: early container runtimes had documented ways for a process to “escape” to the host or other containers.
- Guidance: containers are great for isolating TRUSTED code. For untrusted/multi-tenant workloads, dig deeper into current container security practices.

Windows containers

- Fully supported since Windows Server 2016.
- Challenge: Windows OS images are big (early Nano Server > 1GB vs. a few MB for Alpine Linux) — though image-layer caching helps.
- Offers a choice of isolation level: standard process isolation (like Linux), OR Hyper-V isolation — each container gets its own lightweight VM, for stronger isolation of untrusted code.

Docker: The Toolchain That Made Containers Mainstream

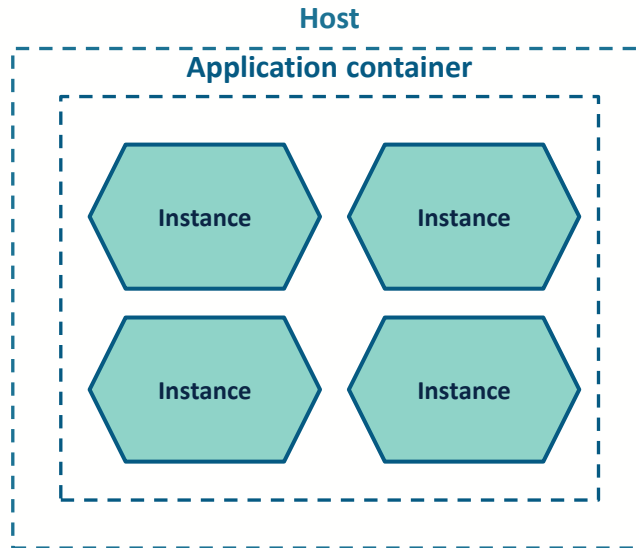


- Before Docker, containers existed but were fiddly: no standard image format, manual networking, no registry.
- Docker added:
 - The IMAGE abstraction — a portable, versioned build artifact for your service
 - Container provisioning and networking, handled for you
 - A REGISTRY concept for storing and sharing images
- Result: a generic toolchain for managing instances, regardless of whether the microservice is written in Go, Python, Node, Java, ...
- Local dev benefit: run the SAME image locally (via Docker Desktop) that you'll run in production — far lighter than running full VMs via Vagrant.
- Docker Swarm / Swarm Mode tried to solve multi-machine orchestration — but Kubernetes won that space.

Option 4: Application Containers



Several services share the same runtime instead of running in separate containers.



❖ Pros

- ☒ Uses less memory → only one JVM is running.

❖ Cons

- ✗ All services must use the same technology (e.g., Java).
- ✗ Services are less independent → they share one runtime.
- ✗ Restarting one service can affect the others.
- ✗ Harder to monitor and debug each service separately.
- ✗ Encourages shared state, which goes against microservice design.

Option 5: Platform as a Service (PaaS)



- PaaS takes a build artifact (a JAR/WAR, a Ruby gem, a container image) and runs it for you — you rarely think about “machines” at all.
- Some PaaS platforms also provide databases, queues, etc. as managed add-ons — **Heroku** set the early benchmark for developer-friendly PaaS.
- **The trade-off:**
 - When it works, it works GREAT — huge reduction in operational overhead
 - When it doesn't fit your use case, you often can't “get under the hood” to fix it
- Trend: standalone PaaS growth slowed; instead, cloud providers now offer turnkey serverless building blocks (managed queues, databases, storage) that you mix and match — paving the way for FaaS.

Platform as a Service (PaaS)



You ship code. The platform handles everything else.

What You Do

- 1 Write your app code
- 2 Build an artifact (JAR, container image)
- 3 Push it to the PaaS (e.g. heroku push)
- 4 Done — your app is live

What the Platform Does For You

- ✓ Choose & provision servers
- ✓ Install OS & runtime
- ✓ Configure load balancer
- ✓ Scale up / down automatically
- ✓ Restart crashed instances
- ✓ Manage TLS certificates

PaaS Trade-offs: Great Fit vs. Poor Fit

✓ When PaaS works great

Small/medium teams

No dedicated ops engineer needed.

Standard web apps

HTTP APIs, background workers — PaaS was built for these.

Managed add-ons

Databases, queues, caches — attach with one command.

Fast iteration

Push code → live in seconds. Great for startups.

✗ When PaaS hurts you

Custom OS / kernel tuning

You can't touch the underlying machine. No sysctl, no eBPF.

Unusual runtimes

If your language/version isn't supported, you're stuck.

Long-running processes

Some PaaS platforms time out long HTTP or batch jobs.

Vendor lock-in

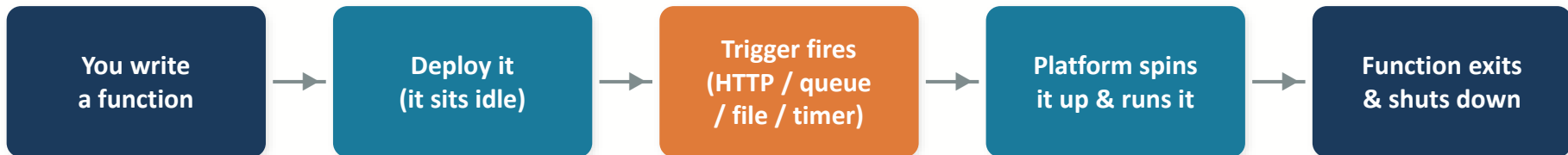
For example Heroku-specific config vars, buildpacks

Option 6: Function as a Service (FaaS)



You deploy a function, not a server.

*"Serverless doesn't mean no servers — it means **you don't think about them.**"*



Pay per execution

You pay only for the milliseconds your function actually runs. If nobody calls it, it costs \$0.



Auto-scaling

The platform runs 1 or 10,000 instances of your function in parallel — you never configure a replica count.



Zero ops overhead

No servers to patch, no load balancer to configure, no daemon to keep alive.

PaaS vs FaaS — What's the Real Difference?



	PaaS	FaaS
What you deploy	A full app / service (always-on process)	A single function (idle until triggered)
Billing model	Per hour the app is running (even idle)	Per millisecond of actual execution
Scaling	You configure replicas or the PaaS scales for you	Platform auto-scales ninstance count: $0 \rightarrow \infty$
Long-running work	✓ Fine — process stays alive	✗ Hard limit: ~15 min max
Startup cost	Always warm — fast response	Cold start delay on first request (ms to secs)
State between calls	✓ In-process memory is fine	✗ Stateless — must use external DB/cache
Best for	Traditional web apps, APIs, long-running tasks	Event-driven jobs, webhooks, burst workloads

FaaS: Know the Limits Before You Commit



Execution time limits

AWS Lambda: ~15 min

Google Cloud: ~9 min

Azure: varies by plan

→ Not suitable for long batch jobs



Cold starts

First request after idle has added latency (100ms–3s). Worst with JVM/.NET.

Mitigate: choose Go/Python/Node, or keep functions warm.



Statelessness

Each invocation starts fresh. Can't store data in memory between calls. Must externalise state to a DB, Redis, or S3.



Blast radius / concurrency

Functions scale to thousands of instances in seconds — this can overwhelm downstream systems (DB, Redis) that can't scale as fast.



Limited resource control

Usually only memory is configurable. CPU, I/O, and network are implied by memory tier — you can't tune them independently.



Language / runtime lock-in

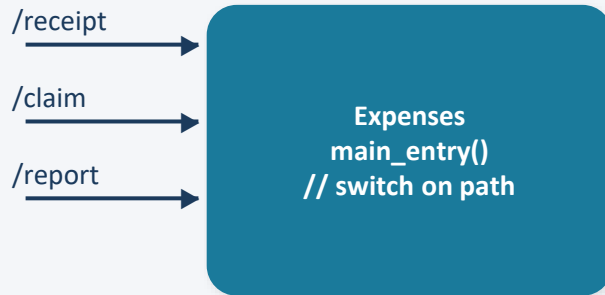
Only provider-supported runtimes available. Azure offers the broadest list; others are more restrictive.

Mapping FaaS to Microservices



Pattern A: Function per Microservice

One function, one entry point that routes internally.



✓ Pros

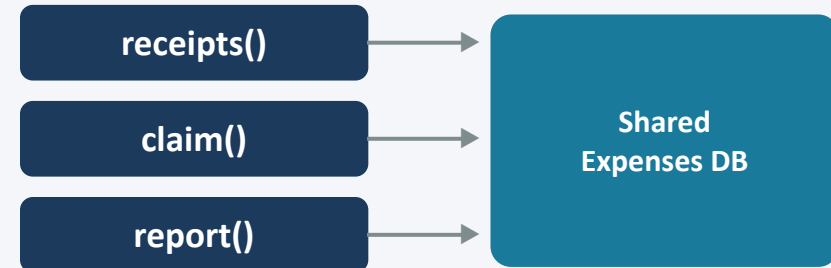
Simple deployment — one unit to version, test, and monitor.
Good starting point when migrating an existing service to FaaS.

⚠ Watch out

Can grow into a monolith-in-a-function if not careful.

Pattern B: Function per Aggregate

Each domain aggregate is a separate deployable function.



✓ Pros

Independent deploy & scale per aggregate. Failures are isolated.

⚠ Watch out

Shared DB is a coupling trap. If you split the DB later, these become true separate microservices — plan ahead.

Deployment Options: Feature Comparison



Option	Isolation	Automation	IaC	Zero-downtime	Desired state
Physical machine	●	○	⊖	○	○
Virtual machine	●	⊖/●	●	⊖/●	⊖/●
Container (+ orchestrator)	⊖	●	●	●	●
Application container	⊖	⊖	⊖	○	○
PaaS	⊖	●	⊖	●	●
FaaS	⊖	●	●	●	●

● Full support

⊖ Partial support

○ No / weak support



Part IV

Kubernetes & Orchestration

By the mid-2010s, container orchestration became the dominant way large organizations run microservices. Kubernetes won that race

Why Container Orchestration?



Containers solve packaging & isolation — but who decides where each one runs?

- At small scale you can place containers by hand
- But as services grow to dozens or hundreds, manual placement breaks down fast
- A crashed container on a failed machine goes unnoticed — and unrecovered
- Services move around, making it impossible to hardcode addresses
- Rolling out a new version without downtime becomes a manual nightmare

An orchestrator answers:



Which machine should this container run on?



What if that machine — or the container — dies?

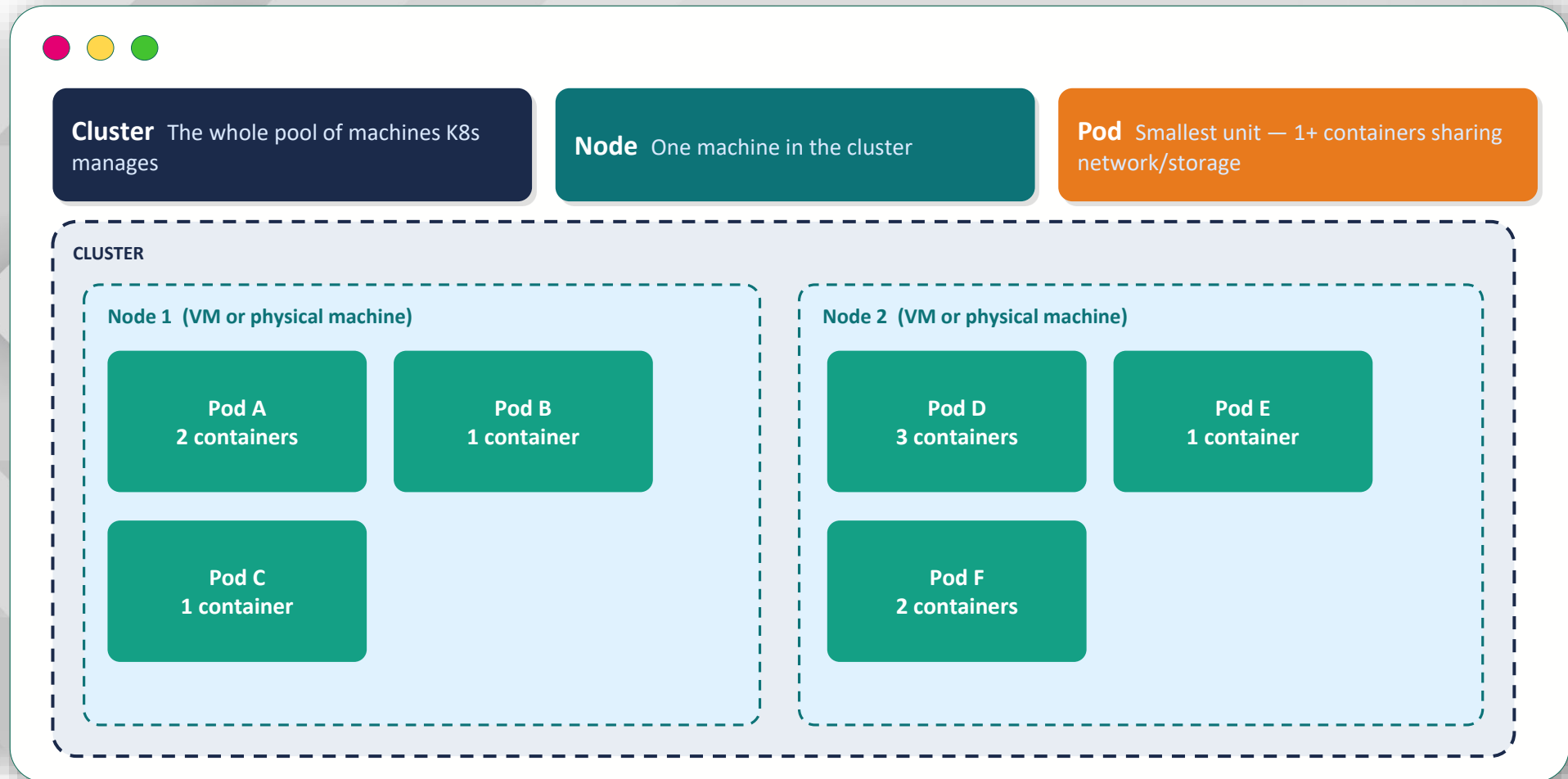


How do containers find & talk to each other as they move?



How do I roll out a new version with zero downtime?

Kubernetes Building Blocks



What Is a Pod? (The Atomic Unit)



POD

Shared Network Namespace (same IP address)

App Container

Your main service

Sidcar Container

e.g. log collector



Shared Volumes (shared file system between containers)

localhost communication

Containers in the same Pod talk via localhost — they share the same IP.



Why group containers in a Pod?

Some processes are tightly coupled — e.g. a web app + a log shipper that must run together. They share the same lifecycle, IP, and disk.



Don't put unrelated things in one Pod

If two services can scale independently or have different lifecycles, they should be separate Pods (and probably separate Deployments).



Pods are ephemeral

A Pod can be killed and rescheduled on a different Node at any time. Never hard-code Pod IPs — use a Service instead.

Deployments & ReplicaSets: Self-Healing Pods



**You write a
Deployment**

desired state:
'3 copies of v2'



**Deployment
manages a**

ReplicaSet



**ReplicaSet
ensures**

exactly 3 Pods
exist at all times



Pod v2 #1

Pod v2 #2

Pod v2 #3

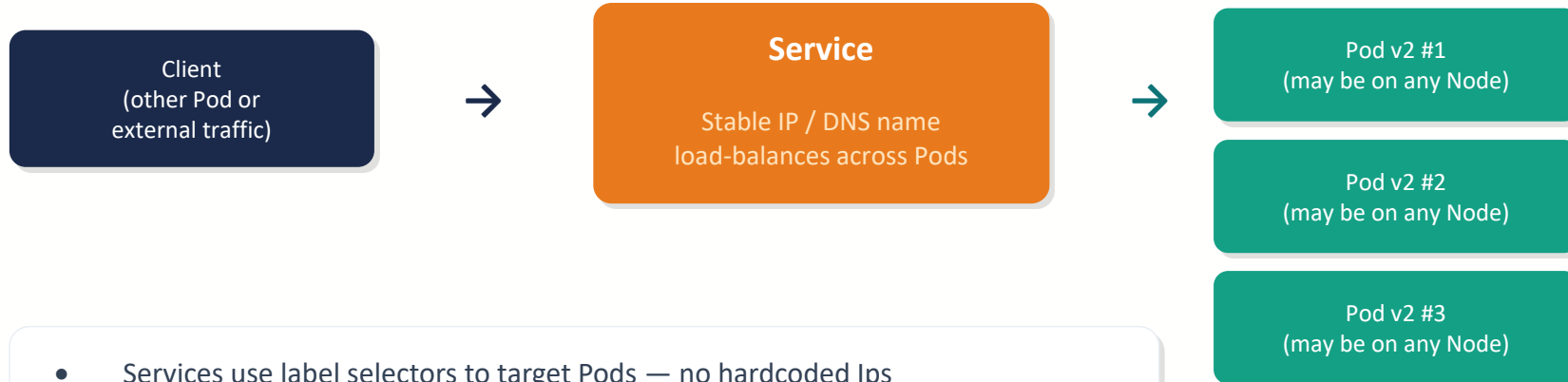


What happens if a Pod dies?

The ReplicaSet controller notices the count dropped below 3 and immediately schedules a new Pod on an available Node. You don't need to do anything — the cluster self-heals toward the desired state.

Services: Stable Addresses for Moving Pods

⚠ Problem: Pods are ephemeral. Their IPs change every time they are killed and rescheduled. You can't hardcode Pod IPs.



- Services use label selectors to target Pods — no hardcoded IPs
- Automatically load-balance traffic across healthy Pods
- ClusterIP (internal), NodePort (node port access), LoadBalancer (external LB), ExternalName (DNS alias)
- Built-in DNS service discovery via `my-svc.namespace.svc.cluster.local`

Rolling Updates: Zero-Downtime Deployments



When you update a Deployment (e.g. new image tag), Kubernetes replaces Pods one-at-a-time, keeping the service alive.



Service always routes to **healthy Pods only** — clients never see a failed request.

You can control rollout speed with **maxSurge** and **maxUnavailable** in the Deployment spec.

Helm: The Package Manager for Kubernetes



Think of Helm like apt or npm — but for Kubernetes apps instead of OS packages or JS libraries.

The Problem Without Helm

- Deploying a database needs 5+ YAML files: Deployment, Service, ConfigMap, Secret, ...
- Every environment (dev/staging/prod) needs its own copy with different values
- Sharing the app with someone means sending them all those files

What Helm Adds

- Bundles all YAMLs into a reusable Chart with templated values
- `helm install my-db bitnami/postgresql` — one command, full setup
- `helm upgrade / helm rollback` — versioned releases
- Share charts on Artifact Hub (like npm registry)

Helm Chart Structure

- `Chart.yaml` — metadata (name, version)
- `values.yaml` — default configuration values
- `templates/` — Kubernetes YAML with `{{ .Values.xxx }}` placeholders
- Override values → `helm install --set replicas=3` (per environment)

Extending Kubernetes: CRDs & Operators



Custom Resource Definitions (CRDs)

- Kubernetes ships with built-in types: Pod, Service, Deployment...
- CRDs let YOU define new types in the Kubernetes API
- Example: define a KafkaCluster resource
- K8s then stores and serves KafkaCluster objects just like built-ins
- The API is the same — `kubectl get kafkacluster` works!

Operators

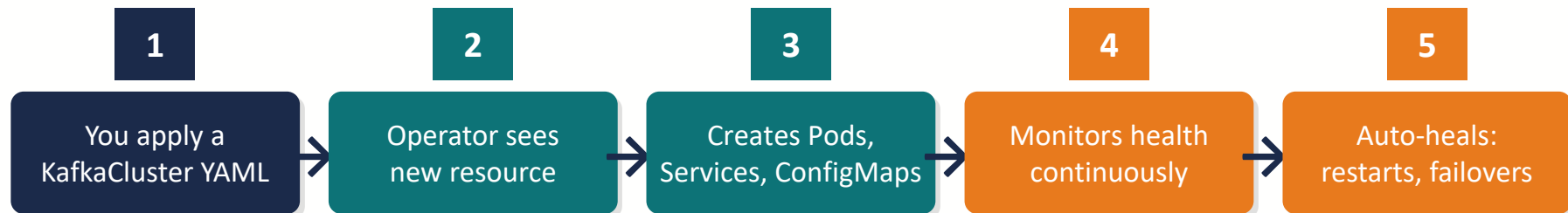
- CRDs store data — but nothing acts on them by default
- An Operator is a controller that watches your CRDs and acts
- It encodes human operational knowledge in code
- Example: a Kafka Operator watches KafkaCluster and automatically handles broker scaling, topic creation, replication, failover
- Effectively: desired state management for complex stateful systems

Together: Helm + CRDs + Operators make Kubernetes a platform for building platforms.

How an Operator Works: The Control Loop



 An Operator runs a control loop: observe → compare with desired state → act. Just like K8s itself — but for your custom resources.



Real-world Operators:

Prometheus Operator — manages monitoring stack, scrape configs, alerting rules

Cert-Manager — automatically provisions & renews TLS certificates

PostgreSQL Operator — handles replicas, backups, failover, upgrades

Should You Use Kubernetes?



K8s is powerful — but comes with real operational complexity. Match the tool to the problem.

✓ Use Kubernetes when...

- Running 10+ microservices that need independent scaling
- Require automatic failover and self-healing
- Need controlled rolling deployments across many instances
- Run multi-tenant workloads with resource isolation
- Have a team with DevOps capacity to manage the cluster

✗ You probably don't need it when...

- A monolith or just 1–3 services
- Early-stage startup — PaaS (Fly.io, Railway, Render) is faster
- Batch or data pipeline workloads → prefer managed services (simpler and lower overhead)
- No DevOps capacity — the overhead will slow you down
- Just learning — start with minikube / kind locally

Key Takeaways



1

Orchestration solves placement, recovery, discovery, and rolling updates — problems containers alone don't address.

2

Cluster → Node → Pod is the hierarchy. Pods are the atomic unit; they share network and storage.

3

Deployments + ReplicaSets give you desired-state self-healing. Services give Pods a stable address.

4

Rolling updates let you ship new versions without downtime — K8s replaces Pods incrementally.

5

Helm packages apps; CRDs extend the API; Operators encode operational knowledge for complex stateful systems.

6

K8s is powerful but complex — match it to the problem. Small teams often do better with a PaaS first.



Part V

Progressive Delivery

Deploying new code and releasing it to users don't have to be the same event.

Separating Deployment from Release



- Deployment:
 - putting a new version of software into a production environment.
- Release:
 - The point at which end users (or other systems) begin experiencing the new version or its functionality.
- Traditionally, deployment and release were often treated as the same event: a team would deploy a new version to production, and users would immediately receive the new behavior.
- Why separate them?
 - The stability of the deployment can be verified before exposing changes to users.
 - Business decisions about when a feature becomes available are decoupled from technical deployment activities.
 - Techniques such as feature flags, canary releases and parallel runs become possible.

Feature Toggles (Feature Flags)



What they are

- A configuration switch checked at runtime: “if (flagEnabled('new-checkout')) { ... }”.
- The same deployed binary can behave differently depending on flag state — no redeploy needed to change behavior.

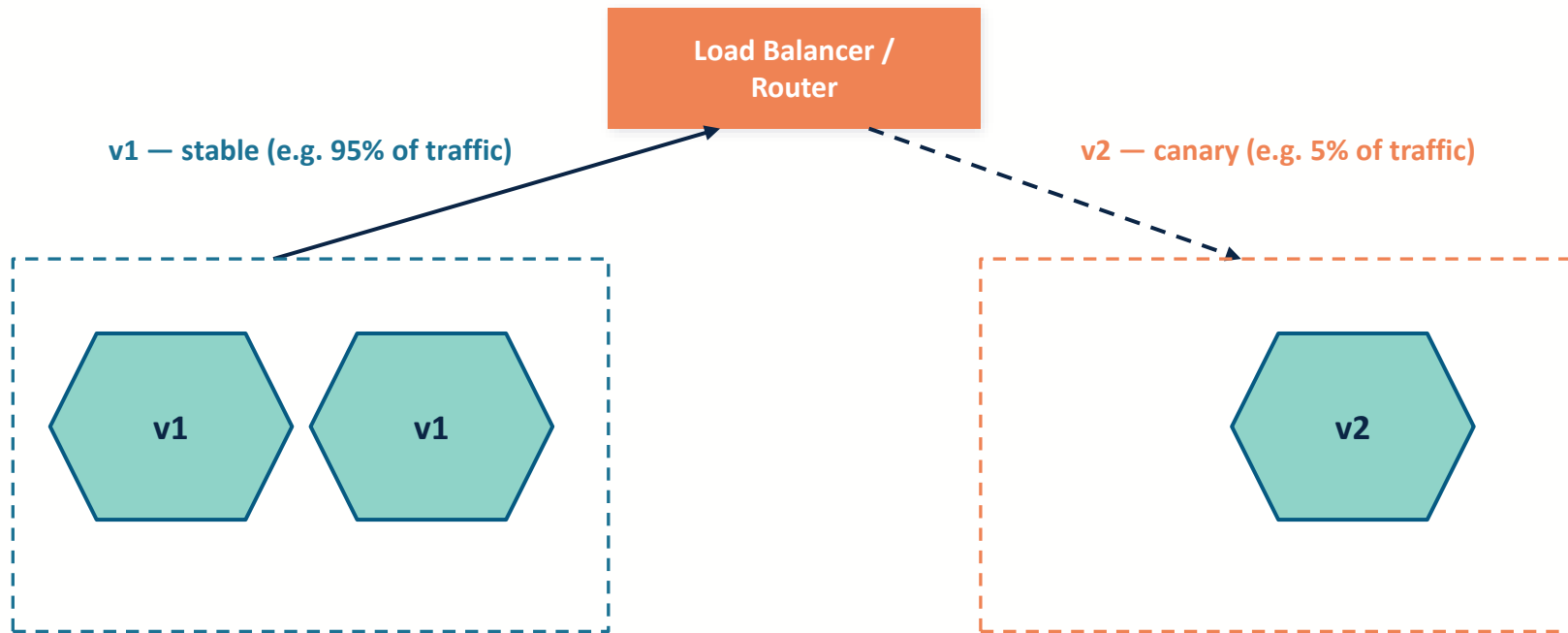
Common uses

- Hide an in-progress feature until it's ready.
- Gradually enable a feature for a subset of users.
- Kill switch — instantly disable a feature causing problems, without a rollback.

The catch — toggle debt

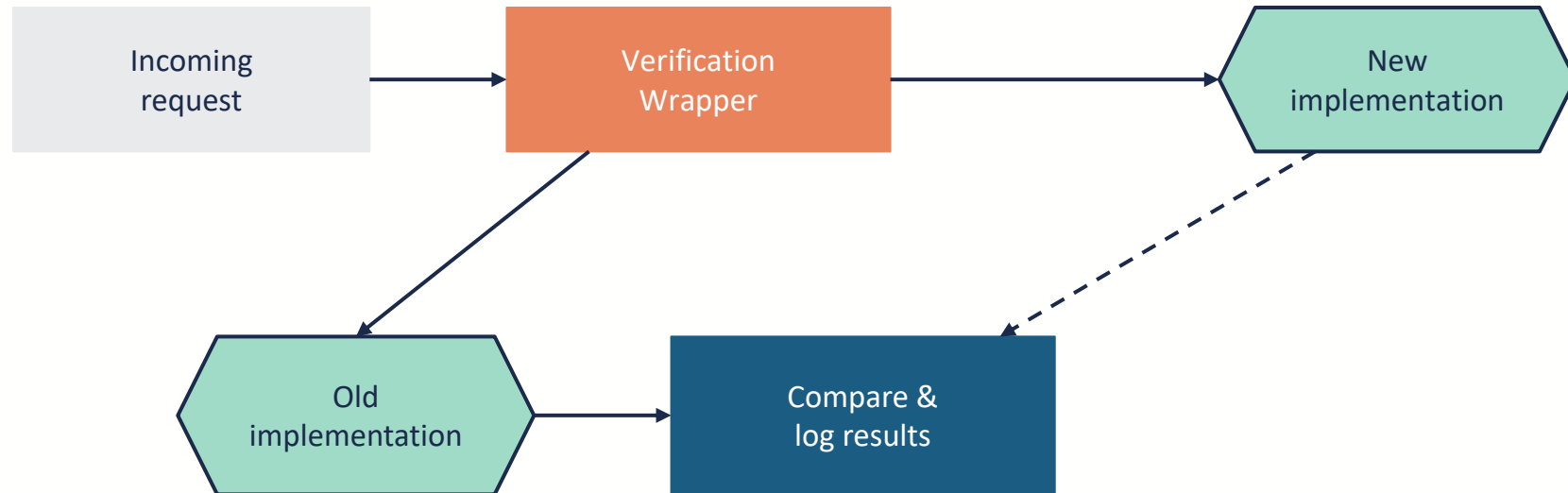
- Every toggle is a branch point: the code paths for 'on' and 'off' both need to work, and both need testing.
- Toggles left in the codebase long after a decision is made become a source of confusion and bugs (“why is this still here?”).
- Rule of thumb: treat toggles as **temporary**. Remove them — and the dead code path — once the rollout **decision** is final.

Canary Release



1. Deploy v2 alongside v1, routing only a small fraction of real traffic to it.
2. Monitor v2 closely — error rates, latency, business metrics — against v1.
3. If healthy, gradually shift more traffic to v2; if not, shift back to v1 — a fast, low-blast-radius rollback.

Parallel Run



Both the old and new implementations execute for the same request, but only the old (proven) result is returned to the caller. The new implementation's result is captured and compared offline, surfacing differences before anyone depends on the new path.

Progressive Delivery: Putting It Together



- Feature toggles, canary releases, and parallel runs all have the same goal: reduce the risk of changes by limiting how many users are affected.
- They can also be used together in a rollout:
 - Deploy with the feature toggle off to verify the new version runs safely without user impact
 - Run a canary release (5%) to test it under real production traffic
 - Operate old and new logic in parallel and compare results on critical paths
 - Gradually enable for all users (100% rollout) once stable
- These approaches support:
 - Principle 4: Zero-Downtime Deployment
 - Principle 5: Desired State Management

A decorative background on the left side of the slide consisting of multiple overlapping, stylized chevron shapes pointing to the right. The chevrons are in various shades of gray, creating a sense of depth and movement.

Part VI

Wrap-Up

Recap: The Five Deployment Principles

**1**

Isolated Execution: Give each service instance its own runtime environment — avoid noisy-neighbor problems and conflicting dependencies.

2

Focus on Automation: Automate builds, tests, and deployments — manual processes don't scale to many services.

3

Infrastructure as Code: Define environments in version-controlled, repeatable definitions — not manual configuration.

4

Zero-Downtime Deployment: Deploy without interrupting users — enables frequent, low-risk releases.

5

Desired State Management: Declare what should be running; let a system continuously reconcile reality to match.

Recap: The Six Deployment Options



Option	Best for	Watch out for
Physical machines	Maximum isolation & control; specialized hardware needs	Slow provisioning, high cost, poor elasticity
Virtual machines	General-purpose workloads needing OS-level isolation	Resource overhead per VM, slower startup than containers
Containers	Most microservices — lightweight, fast, portable	Shared kernel — weaker isolation than VMs; needs orchestration at scale
Application containers	Organizations standardized on a single runtime (e.g. Java app servers)	Coupled deploys/restarts across co-located apps; deployment-model lock-in
PaaS	Small teams wanting to focus on code, not infra	Less control; potential vendor lock-in; may not fit unusual workloads
FaaS	Event-driven, short-lived, highly variable workloads	Cold starts, execution-time limits, statefulness, concurrency/blast-radius risk

Textbooks

- Sam Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2021.



Thank You!

Questions?